

SOFTWARE REQUIREMENTS SPECIFICATION

for

ShopSphere

A responsive React e-commerce frontend with product browsing, cart management, filters, and checkout using the Fake Store API.



Prepared By:

Om Upadhyay

Under the Guidance of:

Mr. Gautam Mukherjee

(Lecturer)

Department of Computer Science & Engineering

University Bachelor of Technology, GLA University, Greater Noida

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, and Abbreviations

1.4 References

1.5 Overview

2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes and Characteristics

2.4 Operating Environment

2.5 Design and Implementation Constraints

2.6 User Documentation

2.7 Assumptions and Dependencies

3. Specific Requirements

3.1 External Interface Requirements

3.2 Functional Requirements

3.3 Non-Functional Requirements

4. System Features

5. Data Requirements

6. Appendices

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the ShopSphere e-commerce frontend application. It will describe the functionality, constraints, and requirements of the system. This document is intended to be used by the development team, project stakeholders, and evaluators.

1.2 Scope

ShopSphere is a responsive e-commerce frontend application designed to simulate a realistic online shopping experience. The application allows users to browse products, search and filter products, view product details, add products to a shopping cart, modify quantities, remove products, and review an order summary before checkout.

The application fetches product information from the Fake Store API and provides a complete frontend shopping workflow including cart management, quantity handling, price calculations, search, filtering, and persistence using localStorage.

This project demonstrates modern React development concepts including component-based architecture, global state management, routing, API integration, derived state, localStorage persistence, and responsive UI design.

1.3 Definitions, Acronyms, and Abbreviations

SRS - Software Requirements Specification

API - Application Programming Interface

UI - User Interface

SPA - Single Page Application

React - A JavaScript library for building user interfaces

TypeScript - A typed superset of JavaScript

Vite - A build tool for modern web applications

Tailwind CSS - A utility-first CSS framework

local Storage - Web storage for key-value pairs

1.4 References

1. ShopSphere Product Requirements Document (PRD.md)
2. ShopSphere Architecture Document (Architecture.md)
3. ShopSphere Design System (Design.md)
4. ShopSphere Development Rules (Rules.md)
5. ShopSphere Development Phases (Phases.md)
6. Fake Store API Documentation - <https://fakestoreapi.com>

1.5 Overview

The remainder of this SRS is organized into the following sections: Overall Description provides a high-level view of the system. Specific Requirements details the functional and non-functional requirements. System Features describes the main features of the application. Data Requirements outlines the data structures and storage. Appendices provide additional supporting information.

2. Overall Description

2.1 Product Perspective

ShopSphere is a self-contained frontend web application that operates as a Single Page Application (SPA). It is designed to be deployed on static hosting platforms such as Vercel or Netlify. The application does not include a backend server and relies on the Fake Store API for product data.

The application follows a component-based React architecture with clear separation of concerns between UI components, pages, global state, API communication, utility functions, custom hooks, and type definitions.

2.2 Product Functions

The main functions of ShopSphere include:

- Display products from the Fake Store API in a responsive grid layout
- Search products by title with debounced input
- Filter products by category
- View detailed product information on a dedicated page
- Add products to a shopping cart
- Manage cart items (increase/decrease quantity, remove items)
- View cart summary with calculated totals
- Persist cart state using localStorage
- Complete simulated checkout process
- Handle loading states and error conditions gracefully

2.3 User Classes and Characteristics

Primary Users:

- Students demonstrating frontend development skills
- Recruiters evaluating React development ability
- Users browsing products in a simulated online store

Secondary Users:

- Developers studying React state management

- Academic evaluators reviewing the project

2.4 Operating Environment

The application is designed to run in modern web browsers (Chrome, Firefox, Safari, Edge) on desktop, tablet, and mobile devices. It is a client-side rendered application with no server-side requirements for the frontend.

Technology Stack:

- Framework: React
- Build Tool: Vite
- Language: TypeScript
- Styling: Tailwind CSS
- Routing: React Router
- State Management: React Context API
- API Client: Axios
- Data Source: Fake Store API
- Persistence: localStorage
- Icons: React Icons
- Notifications: React Hot Toast
- Animation: Framer Motion

2.5 Design and Implementation Constraints

The following constraints apply to the ShopSphere application:

- No real payment processing - checkout is simulated only
- No real user authentication
- No backend server - frontend only
- No real database - data persistence via localStorage only
- Product data comes from the Fake Store API
- Must use the specified technology stack (React, TypeScript, Vite, Tailwind CSS, etc.)
- No introduction of Redux, backend, or database without explicit approval

2.6 User Documentation

The application is self-explanatory with an intuitive user interface. No additional user documentation is required for basic operation.

2.7 Assumptions and Dependencies

Assumptions:

- Users have access to a modern web browser
- Users have a stable internet connection to fetch product data

- The Fake Store API is available and responsive

Dependencies:

- Fake Store API (<https://fakestoreapi.com/products>)
- Browser support for localStorage
- Modern JavaScript and CSS features

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The application shall provide the following pages:

- Home Page: Product catalog with search, category filtering, and product cards
- Product Details Page: Detailed information about a selected product
- Cart Page: Current cart items, quantity controls, and price summary
- Checkout Page: Order review and checkout confirmation
- Not Found Page: Fallback page for invalid routes

The navigation bar shall provide:

- ShopSphere logo/name
- Home navigation link
- Search functionality (where appropriate)
- Cart icon with item count badge

3.1.2 Software Interfaces

The application shall interface with the following external services:

- Fake Store API: REST API for fetching product data
- Base URL: <https://fakestoreapi.com>
- Products Endpoint: /products
- Categories Endpoint: /products/categories
- Single Product Endpoint: /products/{id}

3.2 Functional Requirements

FR-001: Product Catalog Display

The system shall fetch products from the Fake Store API and display them in a responsive grid layout. Each product card shall show the product image, title, category, price, rating, and action buttons.

FR-002: Category Filtering

The system shall allow users to filter products by category. Categories shall be obtained dynamically from the API. Supported categories include Electronics, Jewelry, Men's Clothing, and Women's Clothing. An 'All Products' option shall also be available.

FR-003: Product Search

The system shall allow users to search products by title. Search shall use debouncing (300ms delay) to avoid unnecessary filtering. Search shall ignore case differences and display an appropriate empty state when no products match.

FR-004: Product Details

The system shall provide a dedicated product details page displaying product image, title, category, description, price, rating, quantity selector, and add to cart button. Navigation shall allow users to return to the product catalog.

FR-005: Shopping Cart Management

The system shall allow users to add products, increase quantity, decrease quantity, remove products, and clear the cart. The cart shall prevent duplicate product entries by increasing the quantity of an existing product.

FR-006: Cart Summary Calculation

The system shall calculate and display total item quantity, item subtotal, cart subtotal, tax, shipping, and grand total. All totals must be derived from the current cart state with no hardcoded values.

FR-007: Cart Persistence

The system shall store cart state in browser localStorage. On application load, the system shall read existing cart data, validate the stored structure, restore valid cart data, or use an empty cart if no valid data exists.

FR-008: Checkout Process

The system shall provide a checkout page with order review displaying products, quantities, item prices, subtotal, tax, shipping, and grand total. The checkout action shall simulate order completion and display a success state.

FR-009: Loading States

The system shall provide appropriate loading interfaces while API data is being retrieved, including skeleton loaders, loading indicators, and disabled states where appropriate.

FR-010: Error Handling

The system shall gracefully handle API failures, invalid API responses, empty product results, invalid product IDs, invalid localStorage data, and unknown routes with meaningful user-friendly error states.

3.3 Non-Functional Requirements

NFR-001: Performance

The system shall avoid unnecessary API requests, use debounced search, apply appropriate React memoization, and lazy-load suitable pages where practical.

NFR-002: Responsiveness

The system shall work on desktop, tablet, and mobile devices with responsive layouts that adapt to different screen sizes.

NFR-003: Accessibility

The system shall use semantic HTML, accessible buttons, proper labels, keyboard-friendly controls, appropriate ARIA attributes, and visible focus states.

NFR-004: Maintainability

The system shall use reusable components, clear naming, separation of concerns, TypeScript types, custom hooks, and centralized API logic.

NFR-005: Usability

The system shall provide a modern, clean, trustworthy e-commerce interface with intuitive navigation, clear pricing, and simple cart interaction.

4. System Features

4.1 Product Display Feature

Description: The system shall display products fetched from the Fake Store API in a responsive grid layout.

Inputs: API response from <https://fakestoreapi.com/products>

Processing: Parse API response, render product cards with image, title, category, price, rating, and action buttons

Outputs: Responsive product grid with product cards

4.2 Search and Filter Feature

Description: The system shall allow users to search products by title and filter by category.

Inputs: Search query text, selected category

Processing: Apply debounced search (300ms), filter by category, combine search and filter results

Outputs: Filtered product list, empty state when no matches

4.3 Cart Management Feature

Description: The system shall manage shopping cart state with add, remove, increase, decrease, and clear operations.

Inputs: Product data, quantity changes, cart actions

Processing: Update cart state, prevent duplicates, calculate totals, persist to localStorage

Outputs: Updated cart state, cart badge count, cart summary

4.4 Checkout Feature

Description: The system shall provide a checkout page with order review and simulated completion.

Inputs: Cart items, calculated totals

Processing: Display order summary, simulate checkout process

Outputs: Order confirmation, success state

5. Data Requirements

5.1 Product Data Structure

The system shall handle product data with the following structure:

- id: number - Unique product identifier
- title: string - Product name
- price: number - Product price
- description: string - Product description
- category: string - Product category
- image: string - Product image URL
- rating: object - Rating information (rate: number, count: number)

5.2 Cart Data Structure

The system shall maintain cart state with the following structure:

- cartItems: Array of CartItem objects
- Each CartItem contains: product (Product), quantity (number)
- Derived values: totalItems, subtotal, tax, shipping, grandTotal

5.3 localStorage Persistence

The system shall store cart data in localStorage with the following requirements:

- Use a dedicated storage utility (no scattered localStorage calls)
- Validate stored data before using it
- Never assume localStorage contains valid JSON
- Handle invalid data gracefully by resetting cart

6. Appendices

6.1 API Endpoints

- Get All Products: GET <https://fakestoreapi.com/products>
- Get Product by ID: GET <https://fakestoreapi.com/products/{id}>
- Get All Categories: GET <https://fakestoreapi.com/products/categories>
- Get Products by Category: GET <https://fakestoreapi.com/products/category/{category}>

6.2 State Management

The application uses React Context API for global cart state management. The cart context provides:

- cartItems - Array of items in the cart
- totalItems - Total number of items
- subtotal - Sum of all item prices

- tax - Calculated tax amount
- shipping - Shipping cost
- grandTotal - Final total including tax and shipping
- addToCart(), removeFromCart(), increaseQuantity(), decreaseQuantity(), clearCart() - Cart operations

6.3 Folder Structure

src/

```

├── assets/
├── components/ (Navbar, Footer, ProductCard, ProductGrid, SearchBar, CategoryFilter,
QuantitySelector, CartItem, CartSummary, LoadingSkeleton, EmptyState, RatingStars,
Button)
├── pages/ (Home, ProductDetails, Cart, Checkout, NotFound)
├── layouts/ (MainLayout)
├── context/ (CartContext.tsx)
├── hooks/ (useCart.ts, useProducts.ts, useDebounce.ts)
├── services/ (api.ts)
├── utils/ (calculations.ts, storage.ts, currency.ts)
├── types/ (product.ts)
├── routes/ (AppRoutes.tsx)
├── App.tsx
├── main.tsx
└── index.css

```

6.4 Coding Standards

- Use TypeScript with meaningful variable and function names
- Avoid 'any' type unless documented
- Prefer small reusable components
- Use functional components with React hooks
- All API requests through the API service layer
- Handle loading, success, and error states for all API requests
- Never expose technical stack traces to users